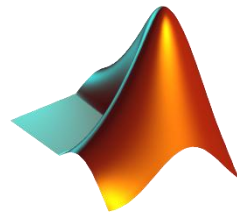


MATLAB

Homework-2

Department of Electrical Engineering
National Taiwan University of Science and Technology
chenyh@mail.ntust.edu.tw



Homework -2

- This NN will be constructed based on the **CIFAR-10** dataset.
- The dataset contains 6,000 color images of size 32×32: 5,000 for training and 1,000 for testing, covering a total of 10 classes.
- Website: <https://www.cs.toronto.edu/~kriz/cifar.html>
- The architecture of the constructed NN is shown below.
- You can find the detail of NN in “**HW2_model.mat**”, the parameter “**convert**”

NN Layers

Deep Learning Network Analyzer

— □ ×

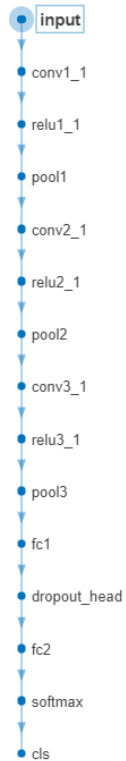
layers

Analysis date: 21-Sep-2025 21:54:45

15 
layers

0 
warnings

0 
errors



ANALYSIS RESULT

	Name	Type	Activations	Learnables
1	input 32x32x3 images with 'zerocenter' normalization	Image Input	32x32x3	-
2	conv1_1 64 3x3x3 convolutions with stride [1 1] and padding 'same'	Convolution	32x32x64	Weights 3x3x3x64 Bias 1x1x64
3	relu1_1 ReLU	ReLU	32x32x64	-
4	pool1 2x2 max pooling with stride [2 2] and padding [0 0 0 0]	Max Pooling	16x16x64	-
5	conv2_1 128 3x3x64 convolutions with stride [1 1] and padding 'same'	Convolution	16x16x128	Weights 3x3x64x128 Bias 1x1x128
6	relu2_1 ReLU	ReLU	16x16x128	-
7	pool2 2x2 max pooling with stride [2 2] and padding [0 0 0 0]	Max Pooling	8x8x128	-
8	conv3_1 256 3x3x128 convolutions with stride [1 1] and padding 'same'	Convolution	8x8x256	Weights 3x3x128x256 Bias 1x1x256
9	relu3_1 ReLU	ReLU	8x8x256	-
10	pool3 2x2 max pooling with stride [2 2] and padding [0 0 0 0]	Max Pooling	4x4x256	-
11	fc1 512 fully connected layer	Fully Connected	1x1x512	Weights 512x4096 Bias 512x1
12	dropout_head 20% dropout	Dropout	1x1x512	-
13	fc2 10 fully connected layer	Fully Connected	1x1x10	Weights 10x512 Bias 10x1
14	softmax softmax	Softmax	1x1x10	-
15	cls crossentropyex	Classification Output	-	-

Weight in convert NN for each layers

convnet

convnet.Layers

convnet.Layers

	1	2	
1	<i>1x1 ImageInputLayer</i>		
2	<i>1x1 Convolution2DLayer</i>		
3	<i>1x1 ReLULayer</i>		
4	<i>1x1 MaxPooling2DLayer</i>		
5	<i>1x1 Convolution2DLayer</i>		
6	<i>1x1 ReLULayer</i>		
7	<i>1x1 MaxPooling2DLayer</i>		
8	<i>1x1 Convolution2DLayer</i>		
9	<i>1x1 ReLULayer</i>		
10	<i>1x1 MaxPooling2DLayer</i>		
11	<i>1x1 FullyConnectedLayer</i>		
12	<i>1x1 DropoutLayer</i>		
13	<i>1x1 FullyConnectedLayer</i>		
14	<i>1x1 SoftmaxLayer</i>		
15	<i>1x1 ClassificationOutputLayer</i>		
16			
17			

Homework -2

- Requirement: **don't use** NN library to finish the NN coded. (predict 、classify 、 activations 、 forward 、 dlarray, dlconv, fullyconnect, maxpool, relu, softmax, batchnorm)
- Can use net.Layers(i).Weights, Bias, Mean, Stride, PaddingSize, padarray, conv2, convn, im2col.
- Input range: 0~1
- Accuracy $\approx 84.5\%$

Homework -2 -Grading

- Requirement: don't use NN library to finish the NN code.
- Due date: 10/8(23:59:59)
- Save the 'xxx.m' file.
- Upload to the Moodle system.
- Grading: Function correctly will be score as = 100.
- Incorrect submission formats will be minus 30 points.
- Function incorrect will be score as 0.

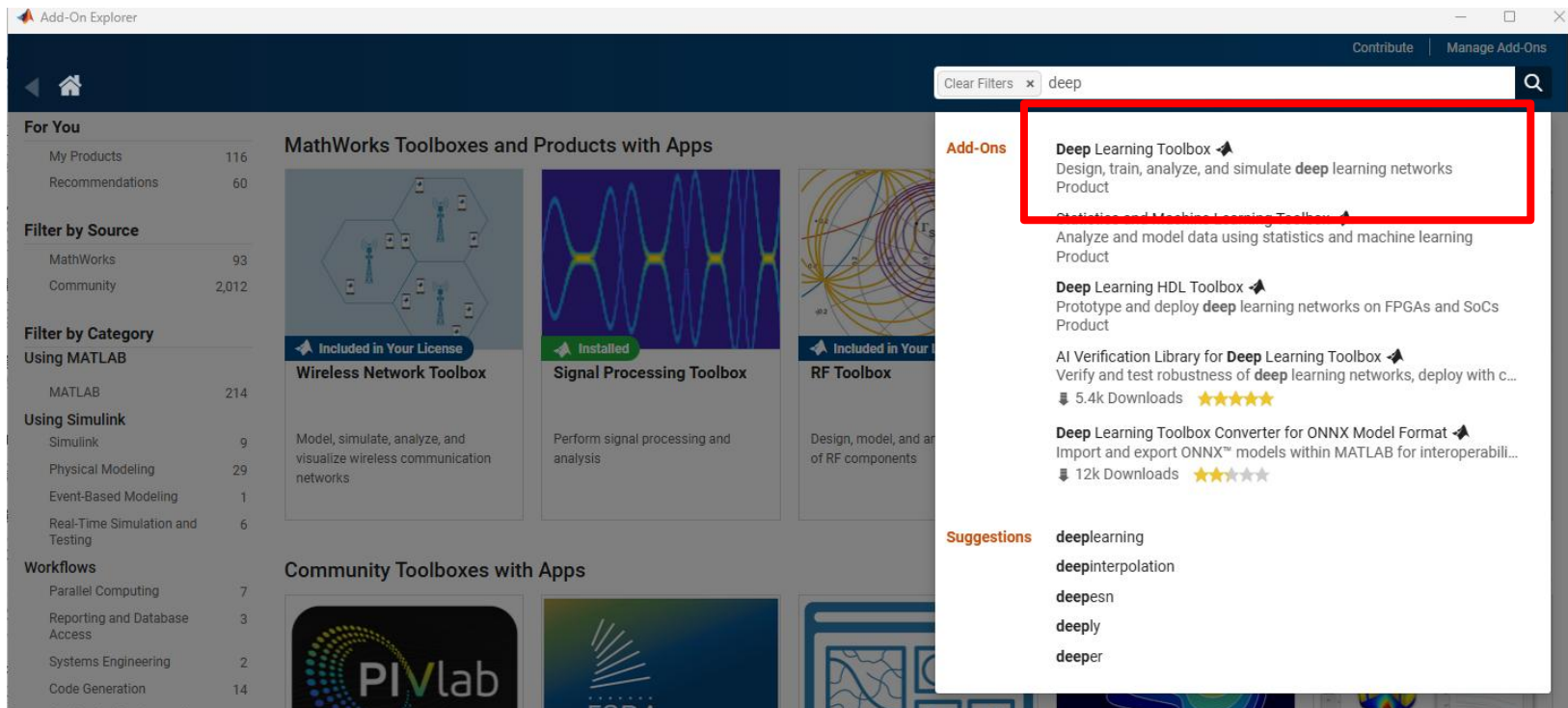
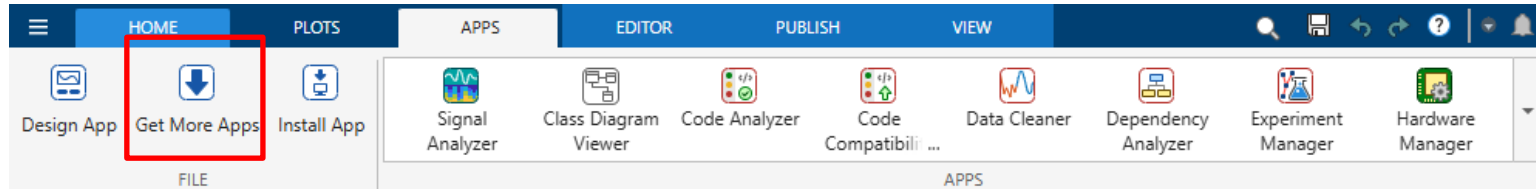
Homework -2 – Submit format

- Top function should be:

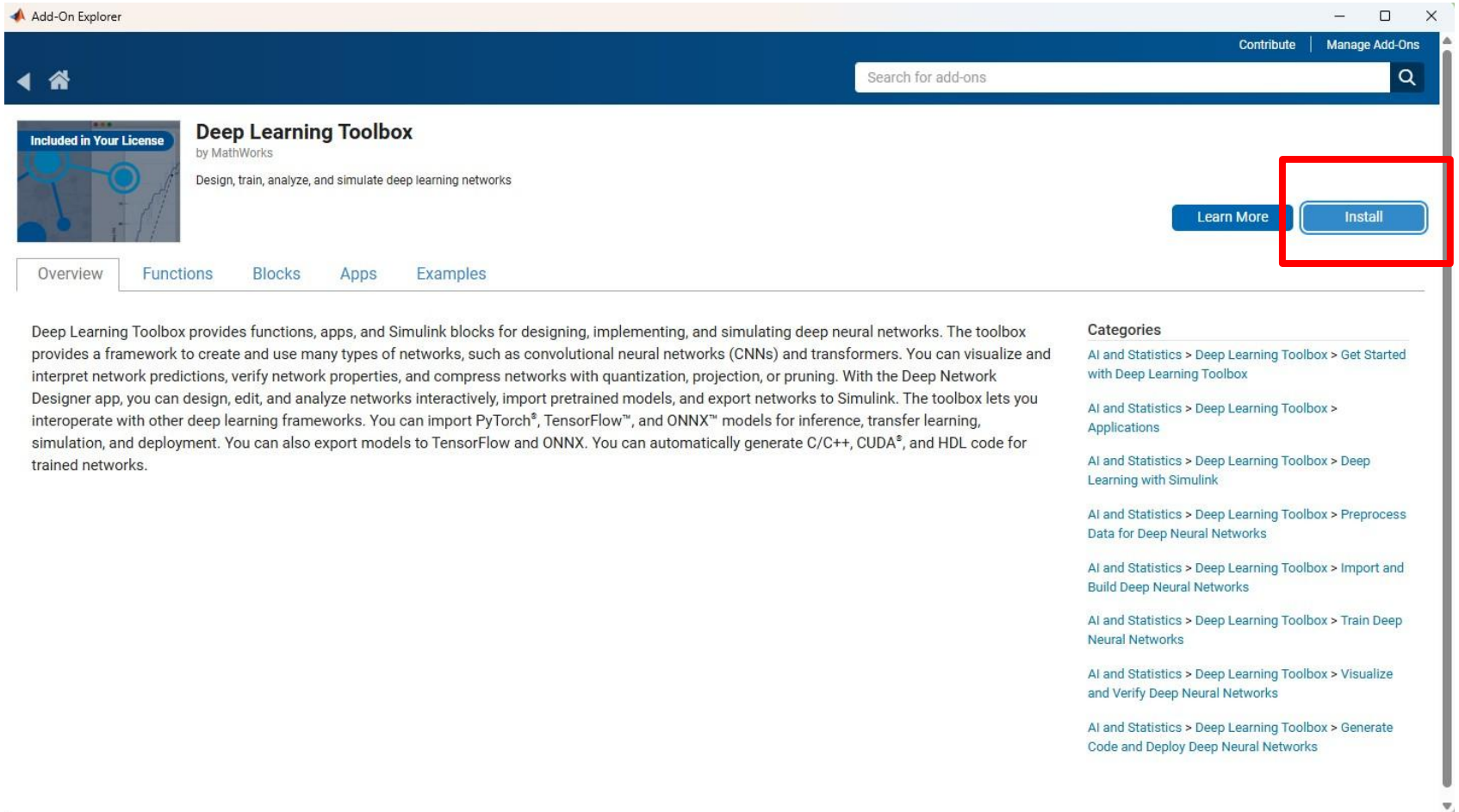
```
function prob = M11XXXXXX(img, net)
```

- And the return type should be 10×1 probability instead of category.
- Design based on the example.m. And don't change the data load and Evaluation part.
- Submit format:
- M11XXXXXX.zip
 - └─ HW2.m

Matlab install Deeplearning Toolbox



Matlab install Deeplearning Toolbox



The screenshot shows the MATLAB Add-On Explorer window. The title bar reads "Add-On Explorer". The top navigation bar includes "Contribute" and "Manage Add-Ons" links, and a search bar labeled "Search for add-ons". The main content area displays the "Deep Learning Toolbox" by MathWorks, with a sub-header "Included in Your License". Below this, it says "Design, train, analyze, and simulate deep learning networks". There are two buttons: "Learn More" and "Install". The "Install" button is highlighted with a red rectangle. Below the buttons are tabs for "Overview", "Functions", "Blocks", "Apps", and "Examples". The "Overview" tab is selected, showing a description of the toolbox and a list of categories on the right.

Deep Learning Toolbox
by MathWorks
Design, train, analyze, and simulate deep learning networks

[Learn More](#) [Install](#)

Overview Functions Blocks Apps Examples

Deep Learning Toolbox provides functions, apps, and Simulink blocks for designing, implementing, and simulating deep neural networks. The toolbox provides a framework to create and use many types of networks, such as convolutional neural networks (CNNs) and transformers. You can visualize and interpret network predictions, verify network properties, and compress networks with quantization, projection, or pruning. With the Deep Network Designer app, you can design, edit, and analyze networks interactively, import pretrained models, and export networks to Simulink. The toolbox lets you interoperate with other deep learning frameworks. You can import PyTorch®, TensorFlow™, and ONNX™ models for inference, transfer learning, simulation, and deployment. You can also export models to TensorFlow and ONNX. You can automatically generate C/C++, CUDA®, and HDL code for trained networks.

Categories

- AI and Statistics > Deep Learning Toolbox > Get Started with Deep Learning Toolbox
- AI and Statistics > Deep Learning Toolbox > Applications
- AI and Statistics > Deep Learning Toolbox > Deep Learning with Simulink
- AI and Statistics > Deep Learning Toolbox > Preprocess Data for Deep Neural Networks
- AI and Statistics > Deep Learning Toolbox > Import and Build Deep Neural Networks
- AI and Statistics > Deep Learning Toolbox > Train Deep Neural Networks
- AI and Statistics > Deep Learning Toolbox > Visualize and Verify Deep Neural Networks
- AI and Statistics > Deep Learning Toolbox > Generate Code and Deploy Deep Neural Networks